

SQL Interview Questions & Answers

Day 3 of 7 — Practice Set (15 Questions)

How to Use This Guide

These are common SQL interview questions framed around this exact dataset. Practice answering out loud in under 60 seconds each — that's roughly what an interviewer expects.

1. What's the difference between WHERE and HAVING?

WHERE filters individual rows before aggregation; HAVING filters groups after GROUP BY/aggregation. E.g., 'WHERE City = "Mumbai"' filters rows, while 'HAVING SUM(Total_Amount) > 100000' filters the grouped result.

2. How would you find the top 3 cities by revenue in this dataset?

```
SELECT City, SUM(Total_Amount) AS revenue FROM perfume_sales GROUP BY City ORDER BY revenue DESC LIMIT 3;
```

3. What's the difference between INNER JOIN and LEFT JOIN?

INNER JOIN returns only rows with matches in both tables. LEFT JOIN returns all rows from the left table, with NULLs where there's no match on the right — useful when you want to keep every customer even if they placed zero orders.

4. How would you calculate a running/cumulative total of revenue by month?

Use a window function: `SELECT month, revenue, SUM(revenue) OVER (ORDER BY month) AS running_total FROM monthly_revenue_cte;` — this avoids self-joins and is the modern standard approach.

5. What's the difference between COUNT(*), COUNT(column), and COUNT(DISTINCT column)?

COUNT(*) counts all rows including NULLs. COUNT(column) counts non-NULL values in that column. COUNT(DISTINCT column) counts unique non-NULL values — e.g., COUNT(DISTINCT Customer_ID) gives the number of unique customers, not total orders.

6. How would you identify duplicate Order_IDs if they existed?

```
SELECT Order_ID, COUNT(*) FROM perfume_sales GROUP BY Order_ID HAVING COUNT(*) > 1; — any Order_ID appearing more than once is flagged.
```

7. What is a CTE (WITH clause) and why use one?

A Common Table Expression is a named, temporary result set defined with WITH that you can reference like a table in the main query. It makes multi-step logic (like computing customer order counts, then filtering by them) far more readable than nested subqueries.

8. How would you find customers who placed more than 5 orders?

```
SELECT Customer_ID, COUNT(*) AS order_count FROM perfume_sales GROUP BY Customer_ID  
HAVING COUNT(*) > 5 ORDER BY order_count DESC;
```

9. What's the difference between UNION and UNION ALL?

UNION combines result sets and removes duplicate rows (slower, involves a sort/dedupe step). UNION ALL keeps all rows including duplicates and is faster — use it whenever you know the sets won't overlap or duplicates are fine.

10. How would you calculate month-over-month revenue growth %?

Use LAG() to pull the prior month's revenue: `SELECT month, revenue, (revenue - LAG(revenue) OVER (ORDER BY month)) * 100.0 / LAG(revenue) OVER (ORDER BY month) AS mom_growth_pct FROM monthly_revenue;`

11. What's a self-join and when would you use one?

A self-join joins a table to itself, typically to compare rows within the same table — e.g., finding pairs of orders from the same customer placed on the same day.

12. How do you handle NULL values in aggregate functions?

Most aggregate functions (SUM, AVG, COUNT(column)) automatically ignore NULLs. Use COALESCE(column, 0) to substitute a default value before aggregating if you need NULLs treated as zero instead of ignored.

13. What's the execution order of a SQL query?

FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT. This is why you can't reference a SELECT alias in a WHERE clause, but you often can in ORDER BY or HAVING depending on the database engine.

14. How would you find the second-highest spending customer?

`SELECT Customer_ID, SUM(Total_Amount) AS spend FROM perfume_sales GROUP BY Customer_ID ORDER BY spend DESC LIMIT 1 OFFSET 1;` — or use DENSE_RANK() for ties.

15. What indexes would you add to this table for faster analysis, and why?

Indexes on Order_Date (for date-range/monthly queries), City and Brand (frequently grouped/filtered columns), and Customer_ID (for repeat-customer lookups) would speed up the most common query patterns in this project.